

A Secure Session Resumption Mechanism for Vehicle-to-Grid Charging

Landon Hadre

*School of Computer Science
University of Windsor
Windsor, Ontario, Canada
hadrel@uwindsor.ca*

Daniel Slobodnik

*School of Computer Science
University of Windsor
Windsor, Ontario, Canada
slobodnd@uwindsor.ca*

Ikjot Saini

*School of Computer Science
University of Windsor
Windsor, Ontario, Canada
ikjot.saini@uwindsor.ca*

Abstract—The Vehicle-to-Grid Transfer Protocol (V2GTP) under ISO 15118 lacks native encryption, leaving EV charging sessions vulnerable to spoofing, session hijacking, and impersonation attacks. An attacker intercepting communications between the Electric Vehicle (EV) and the Supply Equipment Communication Controller (SECC) can resume or hijack paused charging sessions by impersonating valid parties. We propose a blockchain-based solution utilizing Hyperledger, a permissioned blockchain, to secure V2G charging sessions. Our approach uses cryptographic token rotation for session resumption and records of all session events to an immutable ledger which enables tamper-proof audit trails. The SECC validates resume requests by verifying SHA-256 hashed tokens, with each successful resumption invalidating the previous token and issuing a new one, preventing replay and impersonation attacks. We implemented and evaluated our system, using OMNeT++ for network simulation, and the Hyperledger Fabric test network. The results show that integrating blockchain effectively prevents session hijacking while maintaining performance suitable for the real world.

Index Terms—Vehicle-to-Grid, ISO 15118, Session Resumption, Blockchain, Hyperledger Fabric, Charging Security

I. INTRODUCTION

EV adoption is accelerating globally, with millions of electric vehicles now on the road requiring reliable charging infrastructure. The ISO 15118 standard defines how EVs communicate with charging stations using V2GTP. However, V2GTP was designed primarily for functionality rather than security, and it lacks native encryption at the protocol layer. This creates a significant vulnerability: attackers who can observe network traffic between an EV and a charging station can potentially intercept session information. Session resumption is particularly risky because charging sessions can be paused and resumed later, and if an attacker captures the session context, they could potentially hijack or resume someone else's charging session. The primary threats we address are session hijacking, replay attacks, and impersonation. In a session hijacking attack, an adversary intercepts communication between the EV and the SECC and takes over an active or paused session. Replay attacks occur when an attacker captures valid authentication messages and retransmits them later to gain unauthorized access. Impersonation attacks involve an attacker pretending to be either a legitimate EV or charging station. Existing reliability mechanisms like seamless retry focus on improving user experience when connections fail,

but they do not address these security concerns. Seamless retry simply re-initiates failed sessions automatically, which actually could make security worse if not properly protected.

Our work makes three main contributions. First, we propose a secure session resumption mechanism that uses cryptographic token rotation, where each successful resumption invalidates the old token and issues a new one, preventing replay attacks. Second, we implement a permissioned blockchain audit trail using Hyperledger Fabric that records all session events immutably, enabling tamper-proof accountability. Third, we provide a full system implementation with quantitative evaluation showing that our approach prevents attacks while maintaining acceptable performance for real-world deployment.

II. RELATED WORK

Zhdanova et al. demonstrated that weakness in V2G communications can be exploited to manipulate charging behaviour and under certain conditions, threaten local grid stability [1]. Bao et al. performed a threat analysis of the ISO 15118 protocol, identifying vulnerabilities in authentication, information transfer, and the certification hierarchy that could allow adversaries to compromise charging availability or communication integrity [2]. Watt et al. proposed seamless retry to improve charging reliability by automatically re-initiating failed sessions without user intervention, emulating a manual unplug-replug sequence [3]. While this improves operational robustness, it addresses reliability rather than security and does not cryptographically authenticate resumption requests. Our work addresses a fundamentally different problem: preventing malicious session resumption through cryptographic token verification and rotation. Blockchain technology has been explored for various aspects of EV charging infrastructure. Chanthong et al. proposed a blockchain and smart contract system for EV charging payments, enabling peer-to-peer transactions between vehicle owners and charging stations [4]. Chowdhury et al. developed a Hyperledger Fabric-based network for EV charging stations that uses smart contracts to manage authentication, charging sessions, and payment verification [5]. However, these works focus on payment, authentication, or general session management rather than securing the session life cycle against resumption attacks. No prior work secures

session resumption with auditable enforcement, which is the gap our work addresses.

III. SYSTEM AND THREAT MODEL

Our system consists of three main entities. The Electric Vehicle initiates charging sessions and stores resume tokens securely for later session resumption. The SECC manages the entire session life cycle, validate resume tokens using SHA-256 hashing, and records all events to the blockchain. The Charge Point Operator (CPO) monitors charging sessions and can verify audit trails through read access on the blockchain. The SECC connects both an off-chain database for fast mutable state (such as current session status and token validity) and the Hyperledger Fabric blockchain for immutable event logging. Our approach operates under several assumptions. First, the SECC is a trusted entity with write access to the blockchain; we do not consider compromised charging stations in our threat model. Second, the EV has secure local storage for resume tokens, meaning attackers cannot simply read tokens from the vehicle's memory. Third, all communication between the EV and SECC takes place over V2GTP as defined by ISO 15118. These assumptions define the boundaries of our security guarantees.

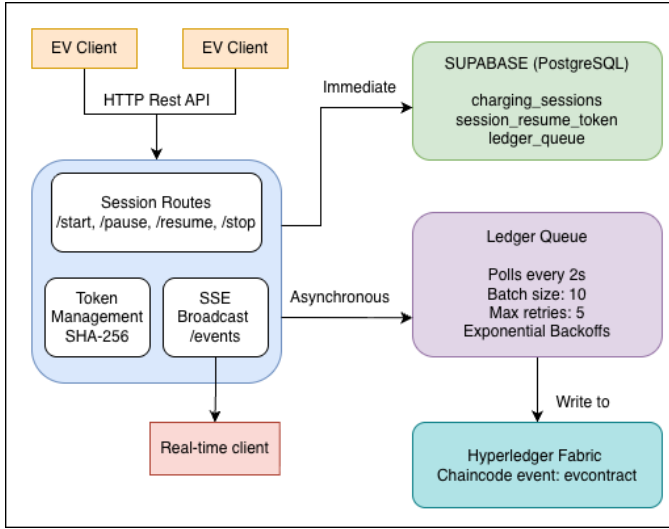


Fig. 1. System Architecture showing EV, SECC, and Blockchain components

A. Threat model

We consider an attacker who can passively eavesdrop on communication between the EV and SECC, capturing all messages exchanged during charging sessions. The attacker can also actively inject or modify messages on the network. The attacker may have access to previously captured session tokens from earlier sessions. Attacks in scope include session hijacking (taking over someone else's session), replay attacks (reusing captured tokens), and impersonation (pretending to be a valid EV or SECC). Attacks out of scope include physical attacks on the charging station hardware, compromise of the SECC itself, backend CPO system attacks and billing or

energy metering fraud. Our focus is specifically on securing the session life cycle against network-based attacks.

IV. SECURE SESSION RESUMPTION

The secure session resumption mechanism ensures that only the legitimate EV that originally started a charging session can resume it after a pause. The core idea is simple: when a session starts, the SECC generates a secret token and gives it to the EV. The SECC keeps only a hash of this token. When the EV wants to resume later, it must present the original token. If the hash matches, the session resumes and a new token is immediately issued, making the old one useless.

Token Generation: When an EV initiates a new charging session, the SECC performs several steps. First, it creates the session record with a unique session identifier, the EV's identifier, and an expiry timestamp (default six hours). Second, it generates a cryptographically secure random token using 256 bits of randomness. This length makes brute-force guessing computationally infeasible. Third, the SECC computes the SHA-256 hash of this token and stores only the hash in its database. The plain text token is never stored. Finally, the SECC returns the plain text token to the EV, which must store it securely for later use. The session start event is then logged to the blockchain.

Session Pause: When an EV disconnects or requests a pause, the SECC updates the session status from active to disconnected. The existing resume token remains valid. The pause event is recorded to the blockchain with a timestamp. The EV retains its resume token during the disconnection period. No new token is issued at pause time because the original token has to yet been used for resumption.

Token Verification on Resumption: When an EV attempts to resume a paused session, it must present its resume. The SECC performs the following verification steps. First, it computes the SHA-256 hash of the presented token. Second, it searches for an unrevoked token record matching the hash. Third, it checks that the token has not expired and that the session is in the disconnected state (not already active or terminated). If any check fails, the resumption is rejected and the failed attempt is logged to the blockchain. If all checks pass, the session resumes successfully.

V. IMPLEMENTATION

A. Simulation Environment

We implement and evaluate our secure session resumption mechanism using OMNeT++ 5.7 [6], a discrete-event network simulator widely used in vehicular and communication systems research. The simulation models a city-scale EV charging network comprising 150 charging points, each rated at 150 kW with a single connector.

The network is built from three module types. *ElectricVehicle* modules implement an 11-state finite state machine covering the complete vehicle lifecycle: arrival, parking, plug-in, session initiation, charging, pause, resume, stop, unplug, and departure. *ChargingPoint* modules act as SECC proxies, managing per-connector state across 10 operational states and

forwarding session operations to the backend via asynchronous HTTP. A central *SimulationController* orchestrates vehicle arrivals according to configurable traffic patterns, assigns parking spots, and collects per-operation latency and throughput statistics.

Inter-module communication uses 16 message types modeled after ISO 15118 V2GTP exchanges, including *SessionInitRequest/Response*, *SessionPauseRequest/Response*, *SessionResumeRequest/Response*, and *SessionEndRequest/Response*. Resume messages carry the cryptographic token required for verification; the corresponding responses carry the rotated replacement token. Modules communicate over *ChargingCable* channels (100Mbps, 1 μ s propagation delay) that represent the wired power-line communication (PLC) link specified by ISO 15118. Because the simulation must interact with a live Hyperledger Fabric network, we use the OMNeT++ real-time scheduler (*cRealTimeScheduler*) at a 1:1 wall-clock ratio so that HTTP round trips and blockchain consensus delays are captured faithfully. Each *ChargingPoint* issues non-blocking HTTP requests to the backend REST API via libcurl with callback-based completion handling, ensuring the simulation event loop is never blocked by network I/O.

The evaluation scenario models realistic arrival patterns over a 10-minute window. During the first two minutes, vehicles arrive at 15 veh/min (ramp-up); a three-minute peak follows at 35 veh/min; the remaining five minutes hold a steady state at 20 veh/min, targeting 150 concurrent sessions. Session durations are drawn uniformly from the range 4–8 minutes. The behavior mix mirrors real-world usage: 94.5% of sessions follow a standard charge-and-depart pattern, 5% pause once and resume, and 0.5% perform multiple pause/resume cycles. To exercise the security mechanisms, we inject one attack per minute from a configurable mix of replay attempts (40%), session hijacking (40%), and identity impersonation (20%).

B. Blockchain Setup

We deploy a permissioned Hyperledger Fabric 2.x network [7] to provide an immutable, auditable record of every charging session event. The network comprises two organizations – *Org1* (SECC operator) and *Org2* (Charge Point Operator, CPO) – each running a single endorsing peer, with a Raft-based ordering service that provides crash fault tolerance. Both peers join a shared channel (*mychannel*) on which the application chaincode is instantiated.

The chaincode, *ev-contract*, is implemented in TypeScript and exposes four transaction functions: *AppendSessionEvent* writes a new event to the ledger, *ReadEvent* retrieves a single event by key, *GetSessionHistory* returns the full event sequence for a given session, and *QueryEvents* supports range-based lookups. Each ledger entry is keyed by a composite key of the form *sess~{sessionId}~{eventType}~{txId}*, enabling efficient prefix queries over all events belonging to a session. The event payload records the session identifier,

event type (*SessionStarted*, *SessionPaused*, *SessionResumed*, *SessionStopped*, or *SessionExpired*), a wall-clock timestamp, and the SHA-256 hash of the current session resumption token. Storing the token hash rather than the plaintext token ensures that a compromised ledger peer cannot recover valid credentials.

A backend service written in Node.js bridges the simulation and the blockchain. It connects to the Fabric network through the *fabric-network* Gateway API using an X.509 identity enrolled under *Org1*'s membership service provider, with peer discovery enabled so that endorsement requests are automatically routed to both organizations. When a *ChargingPoint* module submits a session operation via the REST API, the backend inserts the event into a PostgreSQL-backed queue (Supabase) with status *pending*. A background processor polls this queue every 2s, invokes *AppendSessionEvent* on the Fabric network, and transitions the queue entry to *confirmed* upon successful commit. Failed submissions are retried up to five times with exponential back-off before being marked *failed*.

Token management is integral to the session lifecycle. At session start, the backend generates a 256-bit random token using *crypto.randomBytes(32)*, hashes it with SHA-256, and records the hash on the ledger via *AppendSessionEvent*. The plaintext token is returned to the SECC (and forwarded to the EV) as a *base64url*-encoded string. On each successful resume, a fresh token is generated, its hash is appended to the ledger, and the new plaintext replaces the previous one – implementing the one-time-use token rotation described in Section IV.

C. Integration Details

The simulation and blockchain layers communicate through a RESTful backend that exposes four session endpoints – *start*, *pause*, *resume*, and *stop* – each accepting a JSON payload containing the EV identifier, the session identifier (assigned at start), and, for resume requests, the current resumption token. Responses include the updated session state and, where applicable, a freshly rotated token. Because the OMNeT++ event loop must not block on network I/O, each *ChargingPoint* issues requests through an asynchronous HTTP manager built on the libcurl multi interface [8]. The manager maintains a pool of non-blocking handles; completed transfers are detected by a 1ms poll timer in the *SimulationController*, which invokes the registered per-request callback. Callbacks re-enter the originating module's context (*Enter_Method_Silent*) and drive the connector state machine forward – for example, transitioning from *PENDINGSTART* to *CHARGING* on a successful start response.

On the backend, incoming requests follow a two-phase commit path. The REST handler first persists the session event to a PostgreSQL queue (Supabase) with status *pending* and returns immediately, so the HTTP round trip is bounded by database write latency alone. A background processor then drains the queue in batches of 10 every 2s, submitting each

TABLE I
ATTACK SIMULATION RESULTS

Attack Type	Success Rate	Defence Mechanism
Replay	0%	Token rotation + revocation
Session hijack	0%	Ownership check + hash validation

TABLE II
COMMUNICATION OVERHEAD (10-MIN RUN, 324 OPERATIONS)

Metric	Value
Avg. request size	62 B
Avg. response size	313 B
Total sent	19.9 KB
Total received	99.1 KB

event to the Fabric peer via `AppendSessionEvent`. Upon successful endorsement and ordering, the queue entry transitions to `confirmed` and the in-memory statistics counter is incremented. Failed submissions are retried with exponential back-off ($5s \times 2^a$, where a is the attempt number) up to five times before being marked `dead`. The *SimulationController* monitors blockchain progress by polling the backend’s `/api/queue/status` endpoint every 1s. This endpoint returns both database row counts and in-memory counters. Each poll compares the cumulative confirmed count against the previous value; newly confirmed events are matched in FIFO order to a local queue of pending submissions, yielding per-event wall-clock confirmation latencies that capture the full path from HTTP request to ledger commit. The resulting latency samples are written to a per-run CSV log and summarized in the final report as min, mean, median, P_{95} , P_{99} , and max values.

VI. SECURITY ANALYSIS

To evaluate the resilience of the proposed session resumption mechanism we embed an adversary model directly in the simulation and execute attacks against the live backend during the 10-minute evaluation run. Attacks are injected once per minute from a configurable probability mix: replay (40%), session hijacking (40%), and identity impersonation (20%).

A. Resilience against Replay Attacks

A replay attacker captures a legitimate resumption token – either from the initial `start` response or from a prior `resume` exchange – and later presents the same token to resume the session. The simulation stores every token observed during normal operation and replays one at random when a replay slot is selected. All replay attempts were rejected by the backend (0% success rate). The defence rests on immediate token revocation: when a `resume` request succeeds, the backend hashes the incoming token with SHA-256, matches it against the `session_resume_tokens` table, and atomically revokes the old entry before inserting a freshly generated replacement. Any subsequent use of the revoked token fails the `revoked_at IS NULL` predicate and returns *401 Invalid or revoked token*. Because each token is consumed exactly once, an adversary who captures a token after it has been used gains no advantage.

B. Resilience against Session Hijacking

A hijacking attacker targets an active session belonging to another vehicle. Two sub-techniques are exercised: (i) issuing a `pause` request with a forged EV identifier, and (ii) issuing a `resume` request with a fabricated token. In technique (i) the backend rejects the request because the session query requires both `session_id` and `ev_id` to match, returning *404 Session not found* when the ownership check fails. In technique (ii) the fabricated token hashes to a value absent from the token table, triggering the same 401 rejection as a replayed token. All hijacking attempts were blocked (0% success rate). Table I summarizes the attack outcomes. Token rotation and hash-based validation together defeat both replay and hijacking attacks.

VII. PERFORMANCE EVALUATION

A. Operational Correctness

Over the 10-minute evaluation window, the simulation issued 324 session operations – 213 starts, 13 pauses, 12 resumes, and 86 stops – all of which completed successfully (100% success rate for every operation type). The number of concurrently active sessions peaked at 150 (the configured capacity of the charging network) with a time-weighted average of 99. The HTTP round-trip latency between the simulation and the backend averaged below 2ms, confirming that the asynchronous libcurl pipeline introduces negligible overhead.

B. Communication Overhead

Each session operation requires a single HTTP round trip. Table II reports the aggregate communication statistics. Request payloads average 62 bytes and responses 313 bytes, yielding a per-operation overhead well under 1 KB. The total data exchanged over the entire run is approximately 119 KB, demonstrating that the token-based resumption protocol adds minimal bandwidth cost even at metropolitan scale.

C. Blockchain Confirmation Latency

Because ledger commits are asynchronous, the delay between submitting a session event and its confirmation on the

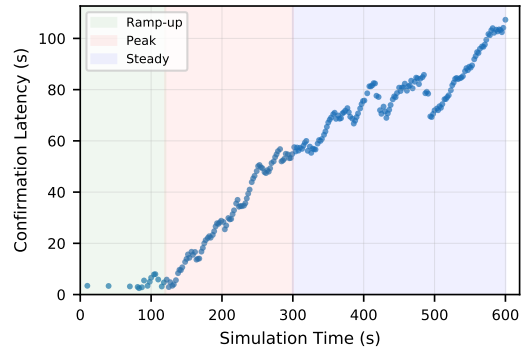


Fig. 2. Per-event blockchain confirmation latency over the 10-minute simulation. Background shading indicates the ramp-up (0–120 s), peak (120–300 s), and steady-state (300–600 s) traffic phases.

Fabric network is the dominant performance cost. We measure wall-clock confirmation latency by recording a timestamp at HTTP submission time and matching each event to its confirmation via FIFO order against the backend’s cumulative confirmed counter.

Fig. 2 plots per-event confirmation latency over the simulation timeline. Under light load (the first two minutes, fewer than 20 concurrent sessions) latency remains below 10 s. As the arrival rate increases and the queue depth grows peaking at 97 pending events by the end of the run latency rises monotonically, reaching a maximum of 107.3 s. Table III presents the distribution. The monotonic increase is expected: the queue processor drains events in batches of 10 every 2 s, yielding a maximum throughput of roughly 5 tx/s, while the peak arrival rate during the spike phase exceeds this limit. In a production deployment the throughput ceiling can be raised by increasing the batch size, adding parallel endorsement pipelines, or deploying additional Fabric peers. Of the 324 events submitted, 229 (70.7%) were confirmed within the simulation window; the remaining 95 were still pending when the simulation terminated, consistent with the growing queue depth observed in the final minutes.

D. Discussion

The results demonstrate that our session resumption mechanism preserves full operational correctness under sustained load, with all 324 operations succeeding and the system reaching the target concurrency of 150 simultaneous sessions. The per-message overhead of the token-based protocol is modest (under 400 bytes per round trip), making it practical for resource-constrained charging infrastructure. The primary performance bottleneck lies not in the resumption protocol itself but in the blockchain confirmation pipeline. The median confirmation latency of 63.7 s is dominated by queueing delay: events wait in the Supabase queue while the background processor serialises Fabric transactions at 5 tx/s. Under light load the end-to-end confirmation time drops below 3 s, showing that the Fabric consensus round itself is fast. Importantly, the confirmation delay is *non-blocking*: the EV receives its session response (including the rotated token) as soon as the REST handler returns, typically within 2 ms. Blockchain confirmation serves as an audit trail and does not gate the charging session, so the queueing latency does not affect the user experience. Scaling the confirmation throughput is straightforward. Increasing the batch size from 10 to 50, parallelising endorsement across multiple goroutines, or deploying dedicated orderer nodes would each independently raise the ceiling. In a production environment these parameters would be tuned to match the expected transaction rate, keeping the queue depth bounded and the tail latency within acceptable limits.

VIII. CONCLUSION

This paper presented a secure session resumption mechanism for ISO 15118-compliant vehicle-to-grid charging sessions, backed by a Hyperledger Fabric permissioned

TABLE III
BLOCKCHAIN CONFIRMATION LATENCY

Percentile Latency	min	median	mean	P95	P99	max
	2.5s	63.7s	57.0s	101.2s	104.0s	107.3s

blockchain. The mechanism centres on one-time-use, cryptographically random 256-bit tokens that are rotated on every successful resume: the backend revokes the consumed token and issues a fresh replacement, while only the SHA-256 hash is committed to the ledger. We evaluated the design with an OMNeT++ simulation of a 150-charger city-scale network connected to a live Fabric deployment. Over a 10-minute run comprising 324 session operations at up to 150 concurrent sessions, every operation completed successfully. Replay and session-hijacking attacks were defeated with a 0% success rate, confirming that token rotation and hash-based validation provide effective defences against the most relevant V2G threats. The communication overhead of the protocol is modest under 400 bytes per round trip and the HTTP response time averages below 2 ms, ensuring that the security layer does not degrade the charging experience. Blockchain confirmation latency, while not on the critical path for session operations, averaged 57 s under peak load due to queueing in the off-chain transaction pipeline. This delay is acceptable for audit-trail purposes and can be reduced by tuning the batch size and parallelising endorsement. Future work will extend the mechanism with ISO 15118-20 Plug-and-Charge certificate binding to strengthen identity verification at session creation, and will evaluate the system on a multi-orderer Fabric topology to characterise confirmation latency under higher transaction throughput.

REFERENCES

- [1] M. Zhdanova, J. Urbansky, A. Hagemeyer, D. Zelle, I. Herrmann, and D. Höffner, “Local power grids at risk: An experimental and simulation-based analysis of attacks on vehicle-to-grid communication,” in *Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC ’22)*. Association for Computing Machinery, 2022, pp. 42–55. [Online]. Available: <https://dl.acm.org/doi/10.1145/3564625.3568136>
- [2] K. Bao, H. Valev, M. Wagner, and H. Schmeck, “A threat analysis of the vehicle-to-grid charging protocol ISO 15118,” *Computer Science - Research and Development*, vol. 33, pp. 3–12, 2018.
- [3] E. Watt, M. Savargaonkar, and P. Gadamsetty, “Recommended practice: Seamless retry for electric vehicle charging,” ChargeX Consortium and Idaho National Laboratory, United States, Tech. Rep. DOE/GO-102024-6447, Dec. 2024.
- [4] N. Chanthong, T. Ruangsakorn, and S. Glomglome, “Blockchain and smart contract payment for electric vehicle charging,” in *2020 17th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*, 2020, pp. 161–164.
- [5] A. Chowdhury, S. S. Shafin, S. Masum, J. Kamruzzaman, and S. Dong, “Secure electric vehicle charging infrastructure in smart cities: A blockchain-based smart contract approach,” *Smart Cities*, vol. 8, no. 1, p. 33, 2025.
- [6] “Omnet++ discrete event simulator,” <https://omnetpp.org/>. [Online]. Available: <https://omnetpp.org/>
- [7] Hyperledger Foundation, “Hyperledger Fabric documentation,” <https://hyperledger-fabric.readthedocs.io/>, 2024, accessed: 2025.
- [8] D. Stenberg, “libcurl – the multiprotocol file transfer library,” <https://curl.se/libcurl/>, 2024, accessed: 2025.